
Cours n° 4

COMPLEXITÉ

Lorsqu'un algorithme est correct, il doit encore, avant d'être programmé et exécuté, satisfaire à deux impératifs en terme de consommation de ressources :

- utiliser un espace en mémoire acceptable, on parle de complexité en espace
- produire la réponse attendue en un temps acceptable, on parle de complexité temporelle

Nous parlons de la complexité d'un algorithme ou de son coût. La complexité (ou le coût) en espace correspond aux tailles des variables utilisées. Elle intervient par exemple dans la manipulation d'images où on recherche pour les algorithmes une complexité constante, c'est-à-dire une écriture d'éléments en mémoire qui utilise un espace constant et réduit autant que possible. Diverses parties d'une machine sont utilisées pour la mémoire et plus les données à conserver ont une taille importante, plus elles sont stockées sur des parties éloignées du processeur avec un temps d'écriture et de lecture qui s'allonge.

Il faut être vigilant avec des algorithmes de tri pour lesquels de nouvelles listes sont construites à chaque appel récursif. Chaque fois que de nouvelles listes sont créées ou que des permutations d'éléments utilisent de l'espace mémoire, on perd en temps d'exécution une partie du bénéfice obtenu par l'algorithme sur le nombre d'opérations à effectuer.

Étudier la complexité temporelle consiste à évaluer le temps d'exécution d'un algorithme en fonction de la taille des données en entrée. Diverses études ont déjà été menées dans les chapitres précédents.

Pour conduire ces études, on évalue un ordre de grandeur du nombre d'opérations élémentaires contenues dans l'algorithme (affectations, opérations arithmétiques, opérations logiques, comparaisons) en supposant qu'elles ont le même temps d'exécution constant. C'est largement suffisant en première analyse. Pour un même problème, ce nombre d'opérations peut être différent suivant les cas. Par exemple trier une liste déjà triée est un cas à priori favorable. Il est important d'étudier le cas le moins favorable, le pire qui puisse se produire, celui qui coûte le plus cher.

Si une machine effectue 10^8 opérations en une seconde, la question n'est pas de savoir si un algorithme effectue $10^8 + 1$ ou $10^8 - 1$ opérations. La question est plutôt d'évaluer un ordre de grandeur du taux d'accroissement du nombre d'opérations en fonction de la taille n des données par exemple pour $n > 10^8$. Si n est multiplié par 2, est-ce que le nombre d'opérations est environ 2 fois plus grand, 4 fois plus grand, 8 fois plus grand ou pratiquement identique?

Avec 10^8 opérations par seconde, si un programme comporte environ 10^{11} opérations et qu'il doit être exécuté mille fois, il faudra attendre la réponse pendant plus de dix jours. Il est donc important de pouvoir choisir l'algorithme le plus performant.

1 TEMPS D'EXÉCUTION

Le temps d'exécution d'un programme dépend de la machine, du langage de programmation, de l'algorithme. Nous ne nous occupons que de ce que fait l'algorithme. Nous supposons que les temps d'exécution d'une affectation, d'une opération arithmétique simple, d'une comparaison sont pratiquement identiques et constituent une unité de base.

Le temps d'exécution d'une suite d'instructions est la somme des temps d'exécution de chaque instruction

Pour un branchement conditionnel :

```
1 if test :
2     instruction 1
3 else :
4     instruction 2
```

le temps d'exécution d'une instruction conditionnelle est inférieur ou égal au maximum des temps d'exécution de chaque cas

Pour une boucle itérative :

```
1 for i in range(p):
2     instruction
```

le temps d'exécution est p fois le temps d'exécution de instructions, si ce temps est constant, éventuellement plus p (le nombre d'affectations pour la variable i) mais on le néglige. Si le temps d'exécution de instructions dépend de la valeur de p , l'étude se mène au cas par cas.

Pour une boucle tant que, l'étude se mène aussi au cas par cas.

L'évaluation du temps d'exécution d'un algorithme se réduit ainsi à une évaluation en fonction d'un nombre n , (entier représentant la taille des données en entrée), du nombre total d'opérations élémentaires noté u_n . Le niveau de complexité correspond au type de croissance de la suite (u_n) .

Suivant les valeurs de l'entrée, u_n peut prendre des valeurs très différentes. Si, par exemple, nous parcourons une liste à l'aide d'une boucle, à la recherche d'un élément, celui-ci peut se trouver en premier et nous sortons de la boucle, c'est le cas le plus favorable. Il peut se trouver à la fin de la liste, c'est le pire des cas.

2 ORDRE DE GRANDEUR

En informatique on utilise le O : $u_n = O(v_n)$ si il existe $B > 0$ tel que $u_n \leq Bv_n$ à partir d'un certain rang (les suites sont à valeurs positives hein).

Cela donne une majoration de l'ordre de grandeur de la complexité.

On a les cas généraux suivants :

Complexité constante : $u_n = O(1)$. Le temps d'exécution est borné, indépendant de n . C'est le cas, par exemple, pour obtenir le premier élément d'une liste.

Complexité logarithmique : $u_n = O(\log n)$. On double le temps d'exécution en élevant au carré la taille des données. Par exemple la recherche dichotomique dans une liste triée.

Complexité linéaire : $u_n = O(n)$. Cet ordre de grandeur peut s'obtenir avec une boucle itérative simple : calcul de la moyenne de n termes, recherche séquentielle dans une liste de n termes.

Notons : on obtient une complexité linéaire en n si le nombre d'opérations à effectuer peut s'écrire sous la forme $\alpha n + \beta$, avec α et β réels, $\alpha > 0$.

Complexité log-linéaire : $u_n = O(n \log n)$. C'est la complexité de certains algorithmes de tri et pour les tris par comparaisons, il est impossible de faire mieux.

Complexité quadratique : $u_n = O(n^2)$. C'est la complexité d'algorithmes construits avec deux boucles imbriquées comme certains algorithmes de tri.

Notons : on obtient une complexité quadratique en fonction de n si le nombre d'opérations à effectuer peut s'écrire $\alpha n^2 + \beta n + \gamma$, avec α, β et γ réels, $\alpha > 0$.

Complexité polynomiale : $u_n = O(n^k)$, $k \geq 2$ fixé. Un algorithme d'une telle complexité n'est utilisable que dans quelques cas particuliers. La complexité est cubique si $k = 3$.

Complexité exponentielle : $u_n = O(2^n)$ ou $u_n = O(k^n)$, $k > 1$. C'est la complexité de certains algorithmes de résolution de problèmes pour lesquels personne n'a encore trouvé d'algorithmes plus performants et qui sont en pratique inutilisables.

Un ordre de grandeur du temps sur un pc standard :

Ordre	nom	$n = 10$	$n = 10^2$	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$
$O(\ln n)$	logarithme	2 ns	5ns	7ns	9ns	12ns	20ns
$O(n)$	linéaire	10ns	0,1 μ s	1 μ s	1 μ s	0,1 ms	1ms
$O(n \ln n)$	log-linéaire	20ns	0,5 μ s	7 μ s	90 μ s	1 ms	20ms
$O(n^2)$	quadratique	0,1 μ s	10 μ s	1 ms	0,1 s	10 s	17min
$O(n^3)$	cubique	1 μ s	1 ms	1 s	17h s	12 J	32 ans
$O(2^n)$	exponentielle	1 μ s	10^{13} a	...	...	...	...

3 EXEMPLES ET CAS TYPIQUES

Une boucle for : il suffit de compter la complexité (maximale éventuellement) d'un passage en boucle, et de multiplier par le nombre de passage en boucle. Généralement la complexité interne est constante, on est alors en $O(n)$

```
1 def f1(n):
2     x = 0
3     for i in range(n):
4         x += 1
5     return x
```

Pour deux boucles for imbriquées on a 2 cas typiques :

Cas : boucles "indépendantes" : Ici chaque boucle est parcouru un certain nombre de fois. Si le coup est constant à l'intérieur de la deuxième boucle on est en $O(np)$.

```
1 def f2(n,p):
2     x = 0
3     for i in range(n):
4         for j in range(p) :
5             x += 1
6     return x
```

Ce qui donne deux sous cas : si p est fixé, c'est donc également un $O(n)$ donc linéaire. Si $n = p$ (ou presque) on est en $O(n^2)$ donc quadratique.

Cas : boucles "dépendantes", le nombre de passage dans la seconde boucle est lié à l'indice de la première.

```

1 def f3(n):
2     x = 0
3     for i in range(n):
4         for j in range(i) :
5             x += 1
6     return x

```

On se place encore une fois où le cout à l'intérieur de la seconde boucle est constant. Les valeurs de i sont successivement $0, 1, 2, \dots, n-1$, on a donc un cout de l'ordre de $\frac{n(n-1)}{2}$, soit $O(n^2)$ une complexité quadratique.

Diviser par 2 : On a "souvent" ce cas, on ne passe pas de i à $i+1$, mais plutôt de i à $i/2$. On est en $O(\log n)$.

```

1 def f4(n):
2     i, k = n, 0
3     while i > 1:
4         i = i//2
5         k = k+1
6     return k

```

Dichotomie : le principe qu'on a vu en TP, on coupe le problème en 2 parties égales, et on traite un sous problème ou les deux.

L'exponentiation rapide, ou la recherche dans un tableau trié est un cas où on traite un seul sous cas, on est alors en $O(\log n)$.

Les tris fusion et rapide (bientôt) sont des cas où on traite les deux sous cas (il faut bien trier les deux morceaux), on est alors en $O(n \log n)$.

Récursivité : le plus simple et souvent de poser le coût et d'établir une relation de récurrence lors de l'appel récursif, on verra ça au chapitre correspondant.

4 COMPLÉMENTS

Est ce qu'on doit donner le nombre exact ou juste un O ?

C'est bien la question. Il faut faire ce que demande le sujet de concours (quand il est clair sur ce point). Néanmoins il faut justifier un minimum. Le nombre exact peut être intéressant : on peut avoir n^2 opérations d'un côté, et $\frac{1}{2}n^2$ de l'autre. Même ordre de grandeur, mais le second est tout de même mieux.

Et la complexité spatiale?

On se restreindra à des exemples, c'est difficile de théoriser car cela dépend fortement de la façon dont sont stockées les données. Le sujet précisera ses attentes.

Un O c'est pas un peu grossier?

Si. Pour citer un collègue "On peut répondre $O(n^n)$, c'est toujours juste".

On a "mieux" que le O , le Θ : $u_n = \Theta(v_n)$ si il existe $A, B > 0$ tels que $Av_n \leq u_n \leq Bv_n$ à partir d'un certain rang.

Cela sera toujours rappelé dans l'énoncé si le sujet en a besoin.

Conclusion bien lire le sujet, et espérer qu'il soit compréhensible.

5 EXERCICES

EXERCICE 1. Déterminer le nombre d'opérations effectuées, puis la complexité du programme suivant

```
1 c = 0
2 while n > 0 :
3     c = c + 1
4     n = n // 10
```

EXERCICE 2. Reprenons la recherche du maximum d'une liste, ci dessous.

- 1) Compter le nombre de comparaison.
- 2) Peut-on donner le nombre exact de d'affectation? Donner un exemple dans le meilleur des cas et le pire des cas.

```
1 def maximum(tab):
2     maxi = tab[0]
3     for val in tab :
4         if val > maxi :
5             maxi = val
6     return maxi
```

EXERCICE 3. Un peu de complexité spatiale - Matrices Creuses

On s'intéresse à deux manières de représenter des matrices creuses, c'est-à-dire des matrices contenant une grande proportion d'éléments nuls :

- La matrice est représentée par un tableau à deux dimensions dont les cases contiennent les éléments.
- La matrice est représentée par un tableau à une dimension. On ne considère que les éléments non nuls de la matrice. Chaque case du tableau contient un triplet (i, j, a) correspondant aux indices de ligne/colonne i, j et la valeur de l'élément a.

- 1) Donner la représentation de la matrice $A = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{pmatrix}$ selon les deux méthodes.

- 2) On veut calculer la somme des éléments d'une matrice. Écrire un algorithme permettant de calculer cette somme pour chacune des représentation.
- 3) Comparer les complexités spatiales (espace mémoire occupé) et les complexités temporelles (nombre d'opérations à effectuer) deux algorithmes : on notera m le nombre d'éléments non nuls.
- 4) Montrer qu'il existe une valeur critique du nombre d'éléments non nuls à partir de laquelle une méthode l'emporte sur l'autre.

EXERCICE 4. Méthode de Hörner

On considère un polynôme $P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$. La liste des coefficients est stockée dans un tableau $a = [a_0, \dots, a_n]$ (attention à l'ordre). On souhaite évaluer ce polynôme en x .

- 1) Ci dessous une proposition. Evaluer la complexité de cette méthode.
- 2) La méthode de Hörner consiste à écrire $P(x) = a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-1} + x a_n) \dots))$.
Mettre cette méthode en pratique et déterminer sa complexité.

```
1 def valuation(x,a) :
2     p = a[0]
3     for i in range(1,len(a)):
4         puis = 1
5         for j in range(1,i+1):
6             puis = puis * x
7         p = p + a[i]*puis
8     return p
```